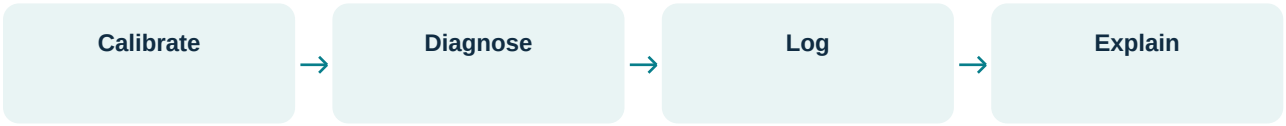
ENGINEERING PORTFOLIO / 02

Instrument Monitor Lab

Calibration, signal-quality checks, and traceable alarm behavior from synthetic current-loop data.



Technical explanation & discussion guide

Prepared for Mridul Batra
21 October 2026

A tested, local simulation project with reproducible evidence. Built with AI assistance; intended for learning, demonstration, and honest portfolio discussion.

Scope: No physical hardware validation, industrial deployment, or safety-system certification.

Guide to this report

Start with the overview, run the default experiment, and then compare the code with the model and test sections. The questions at the end help you explain both what works and where the evidence stops.

01 Overview & practical walkthrough

02 Calibration & quality-aware alarms

03 Architecture & engineering decisions

04 Alarm cause-and-effect matrix

05 Synthetic detection benchmark

06 False alarms & interpretation

07 Default trace & measured output

08 Verification & test matrix

09 Challenges & operating boundaries

10 Local Modbus TCP integration

11 Further work & demonstration

12 Technical questions & further discussion

Quick start

Install Python 3.10+ and double-click **Launch.cmd**. Or run `python src/server.py` from the project folder. No pip install is required for the application.

Run checks with `python -m unittest discover -s tests -v`. Regenerate the default CSV and JSON with `python evaluate.py`.

Optional PDF rebuild: install ReportLab, then run `python docs/build_report.py`. This renders the saved report source and chart; it does not refresh numerical claims automatically.

01 / Overview & practical walkthrough

Instrument Monitor Lab turns a timestamped electrical signal into a calibrated engineering value, checks whether the signal is usable, and records alarm transitions. It is for students and reviewers exploring measurement pipelines, input validation, and explainable diagnostics.

A sensor reading resembles a message in a different language. Calibration translates the electrical value into the quantity we care about. Quality checks then ask whether the message is missing, outside its allowed range, or suspiciously unchanged. Process alarms ask a different question: is the translated value too high or too low?

A complete demonstration

Run the default dashboard. It produces 301 one-second samples with a high excursion, missing data, a frozen signal, a low excursion, and an out-of-range current. Read the calibrated trace and alarm transition log, change the thresholds, and export the results. Upload examples/input-demo.csv to exercise the CSV path.

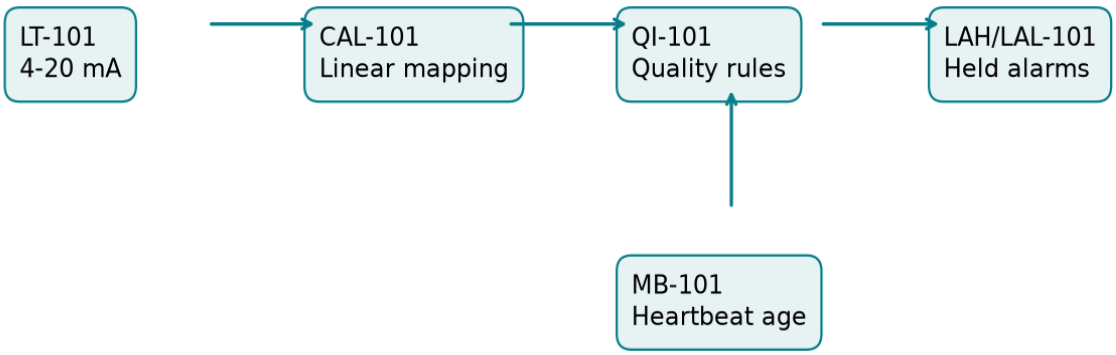
The CSV accepts time_s and raw_mA columns. A blank raw_mA cell means an explicitly missing measurement. Timestamps must be finite, nonnegative, and strictly increasing. The fixed example calibration maps approximately 4.1-19.9 mA onto 0-100 units; do not apply it to a real transmitter without measuring that transmitter.

What is delivered

A least-squares calibration function, deterministic synthetic signal generator, stateful quality checks, hysteretic process alarms, an event log, a responsive dashboard, CSV import and export, JSON export, and 22 monitoring, live-interface and server tests. This is a rules-based diagnostic simulation, not a trained fault-prediction model.

The upgraded monitor adds quality-aware alarm retention, two-sample recovery, spike/noise diagnostics, reference-assisted drift checks, and a local read-only Modbus connection to the tank simulator. The expanded evaluation contains 50 runs across 10 scenarios and five seeds.

02 / Calibration & quality-aware alarms



Educational functional schematic. Tags identify simulated functions; not an approved P&ID.

CAL-101 fits five synthetic calibration pairs: [4.1, 8.05, 12, 15.95, 19.9] mA map to [0, 25, 50, 75, 100] units. This line is an example transmitter conversion, not a physical calibration certificate.

QI-101 marks missing, out-of-range, stale, gapped, frozen, spiky, noisy or reference-inconsistent readings as BAD. While BAD, existing HIGH/LOW states are retained and QUALITY_UNKNOWN is raised. The first good sample is RECOVERING; the second establishes GOOD quality and allows normal hysteresis-based alarm clearing.

LAH-101 raises at 80 units and clears at 77 or below. LAL-101 raises at 20 and clears at 23 or above. New process alarms are not inferred from bad readings; UNKNOWN is the explicit uncertainty state. This avoids falsely declaring recovery after a missing reading.

03 / Architecture & engineering decisions

Actual implementation choices

The application and tests use only the Python standard library. A small browser page uses HTML, CSS, JavaScript, and SVG for interactive plots. This avoids a package installation step and makes the numerical logic easy to inspect. Matplotlib was used only to prepare the static report figure; ReportLab renders this PDF.

The numerical engine is separate from the HTTP handler and browser. Tests call the engine directly, while the dashboard sends JSON to the same function. This prevents the UI from quietly using a second implementation of the science.

The server binds only to 127.0.0.1, rejects unexpected browser origins, limits request size, and writes no uploaded data to disk. It is a single-user demonstration server, with no accounts or database. There is no claim of industrial cybersecurity compliance.

Alternatives considered during implementation

A scientific-stack dashboard was considered, but a standard-library runtime was sufficient for these small deterministic experiments. A database and cloud deployment would add operational work without improving the local learning task. The trade-off is a deliberately small plotting interface and serial request processing.

Retrospective alternatives, not benchmarked

A framework such as Streamlit could accelerate richer dashboards; a production web framework could support authentication and concurrent users. Neither was implemented or benchmarked. Before expanding the audience, reassess deployment, data handling, dependency maintenance, and test coverage.

Transparent deterministic rules were chosen over machine learning because the example fault mechanisms are known and the goal is to explain measurement quality. A fixed calibration makes the demo reproducible, while calibrate() accepts other paired points in code. Least squares is implemented directly to keep the application dependency-free.

Rules remain deterministic and inspectable. Spike detection compares with the previous five-value median; noisy-signal detection uses the standard deviation of successive differences in a ten-value window. Drift requires independent reference data and five consecutive residuals above three units. CSV/live data has no independent reference, so it does not receive a drift-detection claim.

04 / Alarm cause-and-effect matrix

Cause	Quality / process-alarm effect
Missing current or outside 4–20 mA	BAD; retain active HIGH/LOW; reset history and drift counter.
Timestamp gap or heartbeat age >2 s	BAD; hold alarms. Timestamp gap resets stuck timer; stale remains explicit.
Current unchanged within 0.002 mA for 8 s	STUCK; BAD; hold process alarms.
Deviation >12 units from prior 5-point median	SPIKE; BAD. Legitimate step changes may also trigger.
10-value difference-window deviation >3 units	NOISY; BAD. Tail of the window can delay recovery.
Reference residual >3 units for 5 samples	DRIFT; BAD. Requires independent reference.
First clean sample / second clean sample	RECOVERING / GOOD; only GOOD permits process-alarm clear.

No acknowledgement, manual reset, shutdown command, or safety action is implemented. The monitor is advisory. Process alarm state and measurement quality are distinct: a held HIGH with UNKNOWN means the last confirmed state was high and recovery is not yet established.

05 / Synthetic detection benchmark

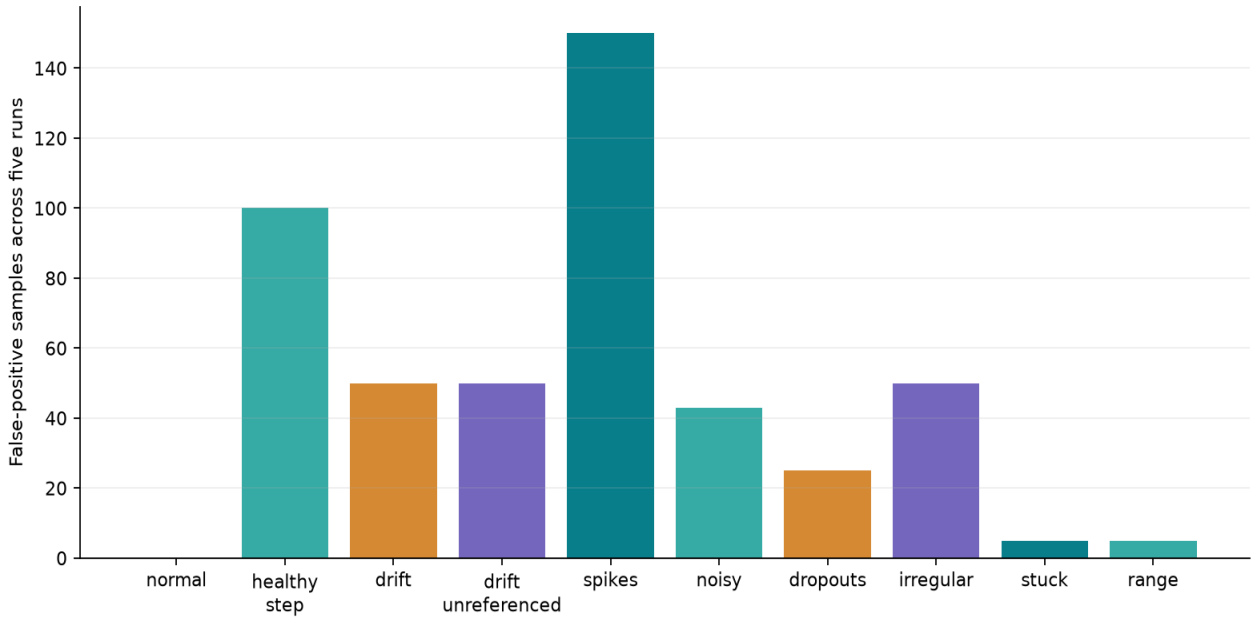
Scenario	Precision	Recall	Delay s
normal	N/A	N/A	N/A
healthy step	0.0	N/A	N/A
drift	0.868	0.656	34.4
drift unreferenced	0.0	0.0	N/A
spikes	0.091	1.0	0.0
noisy	0.919	0.978	2.2
dropouts	0.833	1.0	0.0
irregular	0.5	1.0	0.0
stuck	0.989	0.92	8.0
range	0.99	1.0	0.0

Ten scenarios use five seeds: normal, healthy step, slow drift with/without reference, isolated spikes, noisy signal, repeated dropouts, irregular sampling, frozen signal and range error. Each nominal trace spans 300 seconds. Irregular sampling removes rows; faults are labelled at received gap samples.

Precision = TP/(TP+FP), recall = TP/(TP+FN), evaluated per observed sample. Positive prediction means any bad-quality diagnostic or recovery hold; HIGH/LOW alone is excluded. Event detection must occur within the injected interval. Delay averages only detected events; missed-event counts remain in the CSV. Null means the metric has no denominator or no detected event.

Reference-assisted drift detected all five events with mean delay 34.4 s; unreferenced drift detected none inside the fault interval. Spikes had perfect sample recall but low precision because noise windows and recovery holds extended the flagged state. These are synthetic outcomes, not real-instrument accuracy.

06 / False alarms & interpretation

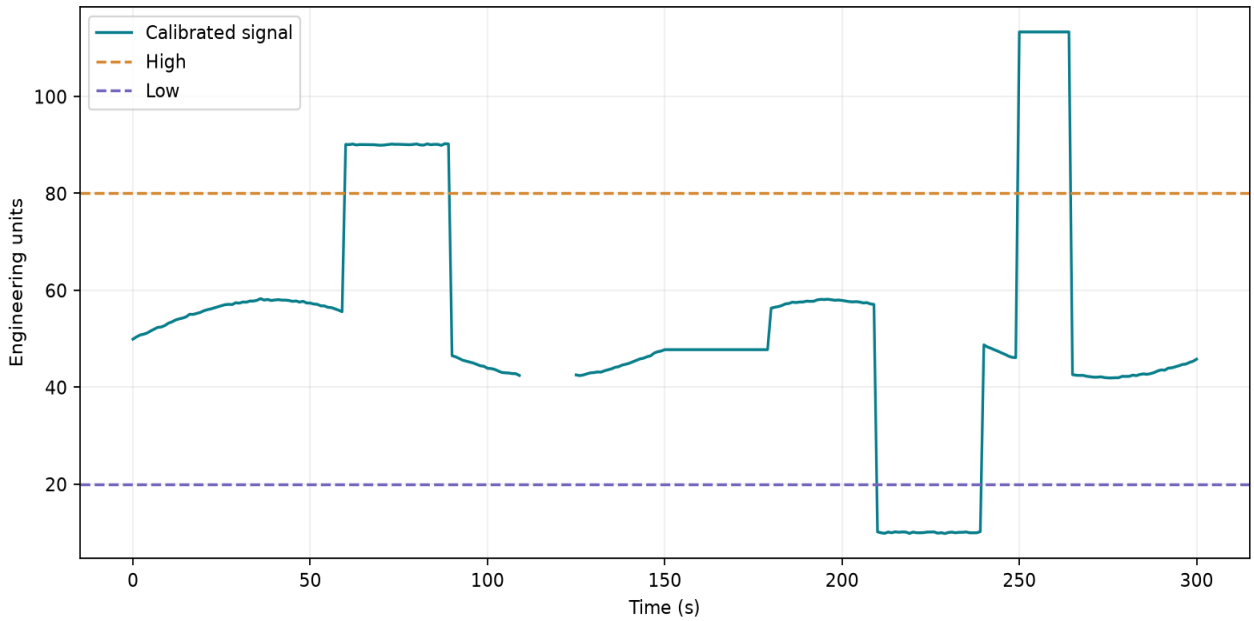


Normal smooth traces produced no false-positive samples in these five seeds. Healthy process steps produced 100 false-positive samples across five runs because transient diagnostics cannot distinguish every real process change from a sensor fault. This is an important boundary, not evidence that the process itself was unsafe.

False-positive samples include intentional recovery hold after a fault ends. They are different from the number of independent nuisance-alarm events. Both seeded event counts and sample-level confusion counts are retained in examples/fault-benchmark.csv. Thresholds were fixed before this benchmark; no optimization against these results was performed.

Useful follow-up: provide process-state context for rapid changes, validate windows against field data, and quantify instrument-specific noise. Reference-assisted drift requires an independent reference; the process truth used in the synthetic benchmark is not available automatically in deployment.

07 / Default trace & measured output



Metric	Measured default
Samples	301
Valid samples (%)	68.44
Alarm raises	20
Reference RMSE	0.282
Calibration slope	6.32911
Calibration intercept	-25.94937

The expanded rules can delay high/low recognition after abrupt steps because quality must recover first. This is visible in the event log. The default plot contains missing and frozen signals, excursions and a range violation; it is one demonstration, not the full benchmark.

08 / Verification & test matrix

Requirement	Expected / actual
Existing behavior	All original tests retained, with revised expectations for intentional quality-state changes. Passed.
Dynamics / alarm behavior	Tank: lag, delay, uncertainty, initialization and seed tests. Monitor: alarm hold, two-sample recovery, repeated dropouts and stale state. Passed.
Data interface	Tank: scaled FC03 reads, invalid address exception, fragmented TCP request. Monitor: frozen heartbeat and disconnect. Passed.
HTTP interface	Valid response, malformed input, missing route, origin rejection, size rejection. Passed.
Numerical / scenario study	Tank: 576 factorial runs + 96 convergence runs. Monitor: 50 synthetic runs. Metrics are results, not universal acceptance certificates.

Each project has 22 passing automated tests, run locally with installed Python 3.14.7. The browser workflow checks desktop/mobile widths, parameter changes, CSV export, and live Modbus stale detection. Test output is saved in docs/test-results.txt. A GitHub Actions workflow is provided but has not run on GitHub.

The comparison matrix includes deliberately difficult conditions. Overflow, non-settling, and false alarms are recorded rather than removed. Passing software tests does not mean every controller setting is acceptable or every fault is detected.

No physical transmitter, PLC, actuator, plant network, or safety function has been commissioned or validated. Browser recording demonstrates a local simulated interface.

09 / Challenges & operating boundaries

Resolved alarm-recovery defect

Previously, invalid data could clear an active process alarm. The revised state machine retains active alarms during BAD and RECOVERING states. Tests cover missing data during HIGH, repeated dropouts, stale data, and two-sample recovery.

Diagnostic ambiguity

Healthy steps can look like spikes, a steady process can look frozen, and slow unreferenced drift can escape detection. Detecting these weaknesses required scenarios beyond the original five scripted conditions. A broader field validation set and process context would be needed to reduce ambiguity.

Communication behavior

A stopped heartbeat becomes stale after two seconds even if TCP still responds. Disconnection produces BAD quality rather than a safe value. The single-user monitor retains at most 10,000 poll records; a full session must be restarted. It reprocesses the bounded history on each poll, suitable for this educational tool rather than an unlimited historian.

Local security boundary

The device supports read-only FC03 on loopback port 15020. It has no write functions, authentication or encryption. It is not intended for remote exposure. The HTTP dashboard also stays on loopback.

10 / Local Modbus TCP integration

Address (zero-based)	Tag / unsigned 16-bit scaling
0	LT-101 true level, metres x 10,000
1	FT-101 inlet flow, m³/s x 1,000,000
2	LT-101 current signal, mA x 1,000
3	LV-101 commanded opening, percent x 100
4	HB-101 one-second heartbeat, modulo 65,536
5	QI-101 device quality; 0 = good

Run python src/modbus_device.py in Tank-Control-Lab, then start the Instrument Monitor dashboard and select Start live polling. The device reads the same tank engine, advancing its precomputed 600-second trajectory with wall-clock time. The trajectory loops for demonstration. No physical PLC is involved.

The client uses Modbus TCP FC03, unit identifier 1, to read six holding registers on 127.0.0.1:15020. Frames validate transaction identifier, protocol identifier, length and byte count. The server handles fragmented reads, illegal addresses and unsupported functions. Device quality is GOOD in this simulation; communication quality is monitored separately.

For a stale-data demonstration, start the device with --freeze-after 6. Its registers and heartbeat stop advancing. The monitor marks STALE after two seconds without heartbeat change. Disconnection also yields unknown quality. A browser check exercised the real local device and stale display.

Protocol reference: Modbus Application Protocol V1.1b3. This implementation is a small read-only subset, not a general-purpose or certified Modbus stack.

11 / Further work & demonstration

Highest-impact improvements

Collect independent reference measurements and healthy/faulty field traces. Set instrument-specific thresholds, sampling periods and uncertainty budgets. Review unknown-state handling with operators before considering any operational use.

One-minute walkthrough

Show calibration and the event log. Start the local tank device, enable live polling, and observe six tags. Demonstrate a frozen heartbeat with --freeze-after 6 and show STALE/QUALITY_UNKNOWN. Import the example CSV, export results, then explain the 50-run benchmark and why healthy steps and unreferenced drift remain difficult. See docs/demo.mp4 and docs/demo-script.md.

12 / Technical questions & further discussion

1. Why calibration rather than raw thresholds?

Engineering units make thresholds interpretable. Calibration accounts for gain and offset; it does not automatically remove random noise or nonlinear error.

2. Why not fill missing data?

Filling would invent measurements and could hide an outage. Missing values are preserved and explicitly flagged.

3. What does hysteresis change?

It gives raise and clear different thresholds. Small oscillations around a boundary therefore do not repeatedly toggle the alarm.

4. Is STUCK proof of failure?

No. It is a heuristic based on low variation. A steady healthy process and a frozen transmitter may be observationally indistinguishable without other evidence.

5. Why is RMSE unavailable for uploaded CSV?

The input schema supplies timestamps and current, not an independent reference. Claiming measurement accuracy without a reference would be unjustified.

6. Why does frozen-signal detection take eight seconds?

The rule waits for enough observed duration to distinguish momentary steadiness from a prolonged plateau. Shorter windows respond faster but increase false alarms.

7. How would you challenge the detector?

Test healthy plateaus, slow drift, coarse quantization, variable sample spacing, and noise just above the tolerance. Report missed events, false events and detection delays separately; do not rely on the demonstration trace alone.

8. What deeper engineering issue remains?

Calibration uncertainty and recovery policy. Ask how reference uncertainty propagates through slope/intercept, and whether two valid samples provide enough recovery evidence for a particular process. The implemented alarm hold is explicit; operational timing still requires domain context.

Implementation references: Python HTTP server documentation (local-server capabilities and production limitations); Python unittest documentation (test discovery and assertions). Accessed 21 October 2026. Model equations, calibration data, and scenarios are defined by this project; they are not measurements from bp assets.